

Spring Batch & Quartz ETL: SFTP → S3 → Snowflake

Executive Summary

This report details the design of a Spring-based ETL module that periodically fetches files from an SFTP server to Amazon S3 and then loads them into Snowflake. The solution uses **Spring Quartz** for scheduling and **Spring Batch** for robust, transactional ETL steps. Key elements include a **metadata table** to track files, Quartz-driven polling logic, and batch jobs with readers/writers for SFTP, S3 and Snowflake. We cover error handling and retry policies, idempotency (avoiding duplicate processing), and scaling (parallel steps, partitioning, clustered schedulers). Security best practices are addressed: SFTP credentials and Snowflake keys stored in AWS Secrets Manager or Vault, S3 bucket policies and IAM roles, and encrypted transfers. Monitoring uses Spring Actuator/Micrometer (e.g.

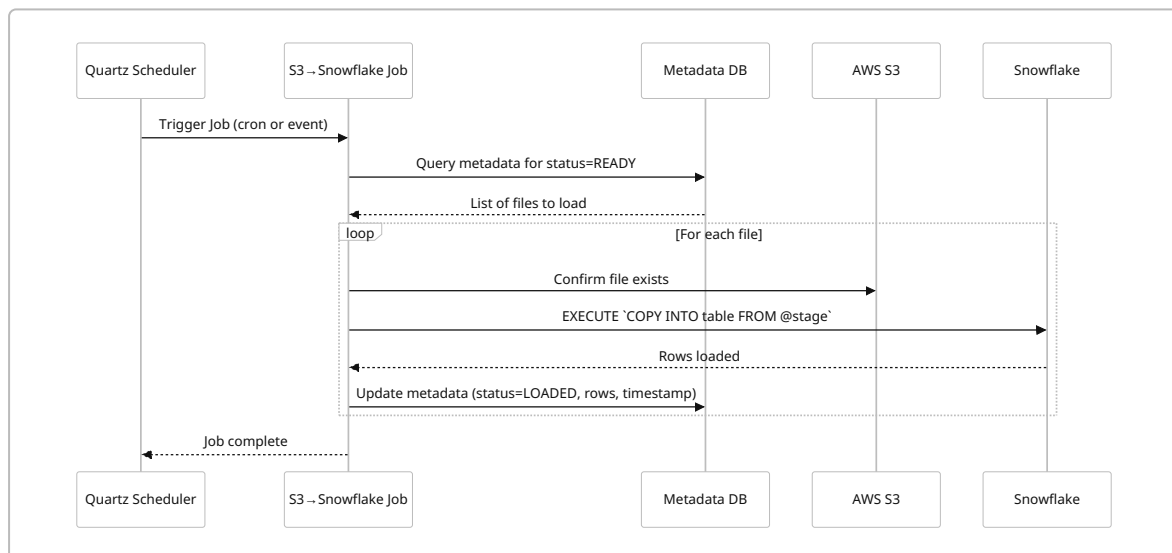
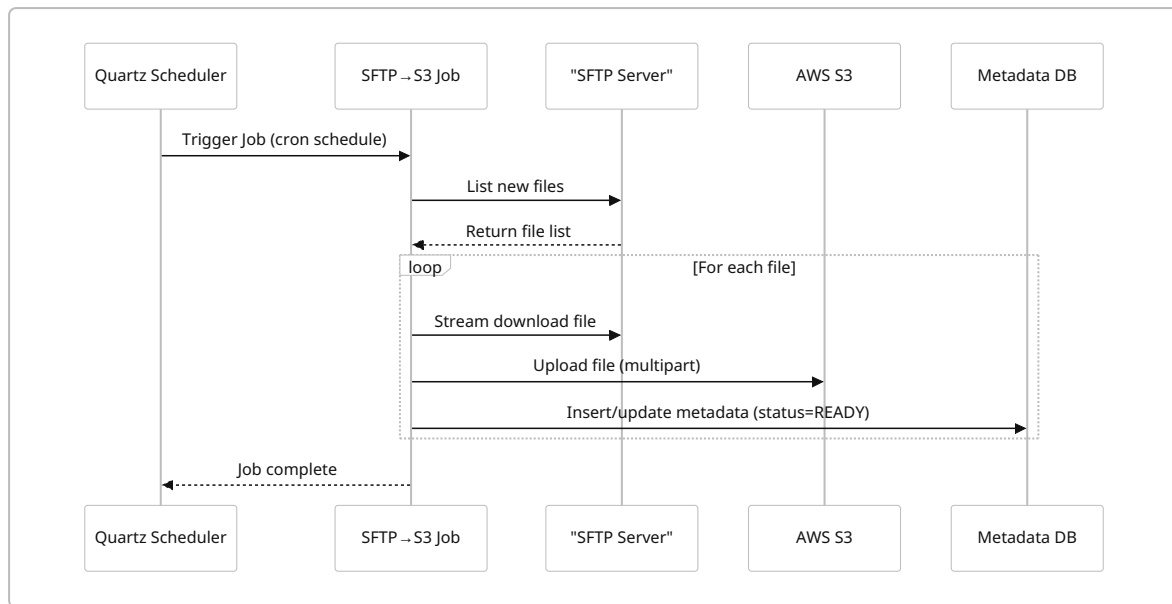
`spring.batch.job` metrics [\[33†L172-L180\]](#)) and log tracking. Deployment options range from a standalone Spring Boot jar to a container/Kubernetes CronJob. We include sample YAML configs and code snippets: Quartz `JobDetail / Trigger` , Spring Batch job/step beans, and readers/writers for streaming from SFTP to S3 and executing Snowflake `COPY INTO` . Recommended libraries: Spring Boot 3.x, Spring Batch 6.x, Spring Cloud AWS 3.x, AWS SDK v2, Spring Integration SFTP 7.x (Apache MINA SSHD), Snowflake JDBC 3.x, Quartz 2.x. Snowflake bulk load uses external S3 stages and `COPY INTO` , favoring ~100–250 MB compressed files [\[22†L267-L271\]](#) . S3 transfers use multipart uploads for >100 MB objects [\[24†L25-L28\]](#) and SSE-KMS encryption by policy [\[25†L169-L174\]](#) . Cost and throughput considerations are discussed, and tables compare alternatives (e.g. polling vs event-driven, push vs pull SFTP, direct streaming vs staged loads). Sequence diagrams (Mermaid) illustrate the workflow, and sample DDL shows a file metadata table.

Architecture & Workflow

The system consists of two main Spring Batch jobs, each triggered by Quartz schedules:

1. **SFTP→S3 Job**: Scheduled by Quartz (e.g. cron) to run at a user-defined frequency. The Quartz trigger launches a Spring Batch job which connects to the SFTP server, lists new files, downloads each (streaming), and uploads it to an S3 bucket. After each upload, it records metadata (filename, S3 location, size, checksum, status) in a database table.
2. **S3→Snowflake Job**: Scheduled (possibly offset from the first job) or triggered after new files arrive. It reads the metadata table for files marked “READY”, then for each file issues a Snowflake `COPY INTO` command from the S3 location into a Snowflake table. After loading, it updates the metadata (rows loaded, status, timestamps).

Each job is a Spring Batch **Job** with one or more **Steps**. For example, the SFTP→S3 job might have a single chunk-oriented step that iterates files; or use Spring Integration to stream files without local temp storage. The Snowflake job may be a Tasklet that executes a batch of `COPY` commands.



Each batch job uses Spring's **JobLauncher** (injected) to run with unique parameters (e.g. timestamp) to prevent duplicate JobInstance conflicts [【17†L237-L244】](#) . The Quartz job itself can be a simple `org.quartz.Job` implementation (or extend `QuartzJobBean`) that autowires a `JobLauncher` and the Spring `Job` bean, invoking `jobLauncher.run(job, jobParameters)` [【17†L237-L244】](#) . Spring Boot's `spring-boot-starter-quartz` auto-configures a `SchedulerFactoryBean`, and Quartz triggers can be defined as beans in Java or YAML [【30†L31-L39】](#) .

Metadata Store: A relational table (e.g. in MySQL, Postgres, or Snowflake) tracks each file. Fields typically include:

- `file_name`, `s3_bucket`, `s3_key`, `file_size`, `checksum` (e.g. SHA-256),
- `status` (`NEW`, `READY`, `LOADED`, `ERROR`),
- `created_at`, `updated_at`,
- `rows_loaded`, `error_msg`.

Sample DDL:

```
CREATE TABLE file_metadata (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  file_name   VARCHAR(255) NOT NULL,
  s3_bucket   VARCHAR(255) NOT NULL,
  s3_key      VARCHAR(1024) NOT NULL,
  file_size   BIGINT,
  checksum    VARCHAR(64),
  status      VARCHAR(20) NOT NULL DEFAULT 'NEW',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  rows_loaded BIGINT,
  error_msg   VARCHAR(1024)
);
```

This table enables idempotency: before processing a file, the batch job checks for an existing metadata record (skip if already DONE) or uses the Spring Batch JobRepository to avoid duplicate JobInstances.

Scheduler & Polling Logic

We use **Quartz** for flexible scheduling. Spring Boot's auto-configuration (via `spring-boot-starter-quartz`) provides a `SchedulerFactoryBean` [30†L31-L39]. Triggers can be defined in Java or `application.yml`. For example, a Cron trigger bean:

```
spring:
  quartz:
    job-store-type: jdbc
    jdbc.initialize-schema: never
    properties:
      org.quartz.threadPool.threadCount: 5

@Bean
public Trigger sftpJobTrigger(JobDetail sftpJobDetail) {
  return TriggerBuilder.newTrigger()
    .forJob(sftpJobDetail)
    .withSchedule(CronScheduleBuilder.cronSchedule("${
    schedule.cron.expression}"))
    .withIdentity("sftp-trigger")
    .build();
}
```

This schedules the SFTP→S3 job per `${schedule.cron.expression}`. Setting `spring.quartz.job-store-type=jdbc` in `application.yml` causes Quartz to persist triggers in a database (allowing clustered schedulers) [30†L37-L44]. In clustered mode, only one instance executes each trigger, enabling horizontal scaling.

The Quartz **job logic** fetches metadata to decide what to process. The SFTP job's trigger simply fires at the defined interval; within the Batch job, we list SFTP files and compare against metadata. Only new

files (`status=NEW` or absent) are downloaded. The Snowflake job's trigger might be on its own schedule (e.g. shortly after the SFTP job) or triggered by a completion event of the first job. Alternatively, one can chain the jobs: the SFTP job's last step could launch the Snowflake job via `JobOperator`.

Spring Batch Job/Step Design

Spring Batch is used for chunk-oriented processing and retries. Each job comprises steps; for instance:

- **SFTP→S3 Step:** A chunk step that reads from SFTP and writes to S3. The reader might be a custom `ItemReader` that lists file names, or a Spring Integration inbound adapter streaming files. The writer uploads to S3. Example pseudocode:

```
@Bean
public Step sftpToS3Step() {
    return stepBuilderFactory.get("sftpToS3")
        .<SftpFile, SftpFile>chunk(1)
        .reader(sftpItemReader())
        .processor(file -> {
            // compute checksum, etc.
            return file;
        })
        .writer(s3ItemWriter())
        .faultTolerant()
        .retryLimit(3)
        .retry(Exception.class)
        .skipLimit(10)
        .skip(Exception.class)
        .build();
}
```

- **SFTP Reader:** Uses Spring Integration or JSch to stream files. For example, a `SessionFactory<ChannelSftp.LsEntry>` bean (Spring Integration SFTP) can list files and a custom reader can `channelSftp.get(remotePath, outputStream)`. Spring Integration 7.x (using Apache MINA SSHD) provides thread-safe SFTP sessions [\[39†L369-L378\]](#).
- **S3 Writer:** Uses AWS SDK v2 `S3Client` or `S3TransferManager`. Spring Cloud AWS auto-configures an `S3Client` bean when `spring-cloud-aws-starter-s3` is on the classpath [\[10†L73-L81\]](#). A writer can call:

```
PutObjectRequest req = PutObjectRequest.builder()
    .bucket(bucket).key(key).build();
s3Client.putObject(req, RequestBody.fromBytes(bytes));
```

For streaming large files, use **TransferManager** with `S3AsyncClient` (CRT) for parallel multipart upload. As AWS recommends, use multipart uploads for files ≥ 100 MB [24†L25-L28] and consider the CRT async client for max throughput [10†L127-L130] .

- **Metadata ItemWriter:** After each upload, update the metadata DB (using `JdbcBatchItemWriter` or a tasklet). Typically, after a chunk commit, a transaction writes the metadata (ensuring exactly-once).
- **Snowflake Load Step:** Could be a Tasklet step that invokes JDBC. For each file (from metadata), execute SQL:

```
COPY INTO my_table
FROM '@my_external_stage/prefix/file.csv.gz'
FILE_FORMAT = (TYPE=CSV FIELD_DELIMITER=',' SKIP_HEADER=1);
```

Snowflake's JDBC driver supports multi-statement execution. It's best to stage all files in one S3 prefix and do one `COPY` per file (or use `PATTERN=` to bulk load). According to Snowflake docs, use an *external stage* for production loads [41†L275-L278] . The stage (created once) points to the S3 bucket/folder. The Spring Batch step collects load results and updates metadata (row count, status).

Error Handling & Retry Strategies

- **Spring Batch retries/skips:** Configure `faultTolerant()` on steps. For transient SFTP or S3 IO failures, use `retryLimit` and specify exceptions. For corrupt records (e.g. parse errors in CSV), use a `skipPolicy` to log and skip faulty lines, possibly sending to a dead-letter list.
- **Transactional commits:** Each chunk commit is atomic – if a write fails, the chunk is retried (or job fails). Database inserts (metadata updates) and file uploads should be in separate transactions, with careful ordering.
- **Idempotency:** Mark files as “READY” only after successful S3 upload. If a job is re-run with the same params, already-completed files (in metadata) are skipped to avoid duplicates. Optionally, rename SFTP files after transfer or move them to an “archive” dir.
- **Snowflake errors:** Use `COPY` options like `ON_ERROR=CONTINUE` or `ABORT_STATEMENT` based on policy. Large load failures can be partial (e.g. skip bad rows), but monitor `information_schema.copy_history` for load statuses.

If any step fails (exception thrown), Spring Batch can retry based on policy, else job goes to FAILED state. In a Quartz job, catch exceptions in the Job class to possibly reschedule or alert (e.g. send email on failure).

Idempotency & Exactly-Once Semantics

To ensure exactly-once processing of each file:

- **Unique Job Parameters:** Include a timestamp or run-id in `JobParameters` so each trigger yields a new `JobInstance`.
- **Metadata Checks:** Before processing, check if `file_metadata.status` is already `LOADED` for that file; skip if so. This prevents reloading.

- **Spring Batch JobRepository:** Spring Batch's repository (tables like `BATCH_JOB_INSTANCE`) also guards against rerunning the same parameter set [【19†L138-L146】](#).
- **Atomic Operations:** Write-to-S3 and DB-update should be logically atomic: e.g. upload file, then insert metadata; if metadata write fails, the upload can be flagged for retry.

Snowflake's `COPY` is idempotent if files are not reloaded twice; keep files in S3 or rename them after loading to avoid duplication.

Concurrency & Scaling

- **Multi-threaded Steps:** Steps can be configured for concurrent processing. For example, a `TaskExecutor` on a step allows multiple threads to upload files in parallel. Spring Batch's `chunk` processing isn't inherently parallel, but you can use a `Partitioner` to divide the file list among threads.
- **Partitioning:** If thousands of files, use `PartitionHandler` to split the file list and process in parallel across threads or even machines. The master step partitions work, and remote or local workers pick tasks.
- **Clustered Quartz:** Setting `spring.quartz.job-store-type=jdbc` and running multiple app instances creates a clustered scheduler – only one instance handles each trigger, avoiding duplicate runs (Quartz's JDBC store provides job locking) [【30†L37-L44】](#).
- **Multiple Workers:** For very large volume, deploy the app to Kubernetes with multiple replicas. Use a distributed lock or rely on Spring Batch's repository + Quartz JDBC store to coordinate (so only one Job runs per schedule). Each worker can pull distinct file subsets if needed.

Security

- **SFTP Credentials:** Store SFTP credentials (username/password or SSH keys) in a secure secrets manager (e.g. AWS Secrets Manager or HashiCorp Vault). Spring Cloud AWS can auto-import secrets via `spring.config.import=aws-secretsmanager:secretName` [【37†L39-L48】](#). At runtime, Spring Integration's `DefaultSftpSessionFactory` uses these credentials (can also use SSH private key resource).
- **AWS IAM:** The application should assume an IAM Role with least-privilege: S3 PutObject/GetObject on the target bucket, and AWS KMS decrypt if using SSE-KMS. In AWS (ECS/EKS/EC2), attach an IAM role to the instance/task. If running externally, provide AWS access keys from Secrets Manager.
- **Snowflake Credentials:** Use a Snowflake user/role with limited rights (only SELECT/INSERT on target tables and USAGE on stage). Store the Snowflake account, user, and auth token in secrets. Snowflake also supports key-pair authentication (private key stored in Vault). Connection is via TLS (JDBC URL).
- **S3 Encryption:** Enforce bucket encryption (SSE-S3 or SSE-KMS) on the bucket. All S3 objects can be encrypted at rest. For transit, HTTPS is used. AWS docs recommend **server-side encryption** by default [【25†L169-L174】](#). Use SSE-KMS for auditability.
- **Network Security:** If running in VPC, use VPC endpoints for S3 and Snowflake (if applicable) so traffic doesn't go over the public internet.
- **Least Privilege:** Grant only necessary permissions in IAM policies (e.g. `s3:GetObject`, `s3:PutObject`, `secretsmanager:GetSecretValue` etc.), per AWS best practices [【25†L213-L222】](#).

Monitoring & Observability

- **Metrics:** Enable Spring Boot Actuator and Micrometer. Spring Batch auto-configures timers/counters under `spring.batch.*`. For example, `spring.batch.job` (duration of jobs), `spring.batch.step` (step durations) and item counts `spring.batch.item`. These can be exported to Prometheus/Grafana or CloudWatch.
- **Logging:** Use SLF4J/Logback to log key events (file transfers, record counts, errors). Include context (job name, file name, attempt). Spring Batch logs each step start/stop; consider adding custom listeners for extra info.
- **Health Checks:** Expose an `/actuator/health` endpoint. The Batch job status isn't automatically reflected, but you can publish Gauge metrics for last run status or hook into Spring Actuator health indicators (custom).
- **Alerts:** Set up alerts on failure conditions: e.g. if a job hasn't run in 2× expected interval, or if many retries occur. Use CloudWatch alarms on custom metrics or log filters.
- **Snowflake Monitoring:** Query Snowflake's `COPY_HISTORY` or `LOAD_HISTORY` views after loads for success/failure metrics.

Deployment Options

- **Standalone JAR:** Package as a Spring Boot fat jar. Can run on an EC2, on-prem server, or any JVM host. Use OS cron or system scheduler to start the app if not using Quartz. Better: let Quartz run inside the app.
- **Docker/Kubernetes:** Containerize the Spring app (JAR). Deploy as a Deployment/CronJob in Kubernetes. Use `ConfigMap` or mounted secrets for `application.yml`. With K8s CronJob, an alternative is to use the Kubernetes scheduler instead of Quartz (running the job as a one-off container). But embedded Quartz gives more control and dynamic scheduling.
- **Cloud Services:** Could run as an AWS ECS Fargate task triggered by EventBridge (cron) or Step Functions. Snowflake loads can be triggered by AWS Lambda (using Snowflake JDBC) or Snowpipe. But for our Spring solution, a container or VM approach is simplest.
- **Configuration:** Application properties (e.g. `application.yml`) store endpoints, paths, cron schedules, etc. Use profiles (`prod`, `dev`) for different environments. Example snippet:

```
sftp:
  host: sftp.example.com
  port: 22
  user: ${SFTP_USER}
  password: ${SFTP_PASS}
  remote-dir: /incoming

aws:
  s3:
    bucket: my-data-bucket
    kms-key-id: alias/myKey

snowflake:
  url: jdbc:snowflake://account.snowflakecomputing.com
  user: ${SNOWFLAKE_USER}
  password: ${SNOWFLAKE_PASSWORD}
  database: MYDB
  schema: PUBLIC
```

```

warehouse: MY_WH

schedule:
  sftp-cron: "0 0/15 * * * ?"      # every 15 minutes
  snowflake-cron: "0 5 1/1 * * ?"  # 5 minutes after the hour

```

Best Practices & Patterns

- **AWS S3:** For large files ($\geq 100\text{MB}$), use multipart upload [\[24†L25-L28\]](#) to improve throughput and resilience. Ensure the AWS SDK v2 is updated (latest release 2.48.x as of 2026). Consider using `S3TransferManager` (v2) or `TransferManager` (v1) for parallel multipart uploads. Set the S3 bucket to enforce encryption (SSE-KMS) in its policy. Use lifecycle rules (e.g. transition or delete old files if needed). Tag or prefix uploaded files for cleanup.
- **Snowflake Bulk Load:** Use **external named stages** pointing to the S3 bucket [\[41†L275-L278\]](#). Define a `FILE_FORMAT` object in Snowflake (CSV, JSON, Parquet etc). Aim for file sizes ~100–250 MB (compressed) [\[22†L267-L271\]](#) to maximize parallel load. Do not copy extremely large single files; split them if needed. Use `COPY INTO` (with `ON_ERROR = ABORT_STATEMENT` or `CONTINUE` as needed). For near-real-time ingestion, Snowpipe (event-driven via S3 notifications) is an alternative (see Snowflake docs).
- **Idempotent Loads:** If using `COPY`, you can tag files as “loaded” (e.g. MOVE them to a different prefix) or set `VALIDATION_MODE=RETURN_ERRORS` to test without ingesting. Optionally, use a Snowflake stream on the target table for CDC or auditing.
- **Cost/Throughput:** S3 charges: storage (\$0.02/GB-month) and requests. Minimize small request overhead by aggregating writes. Snowflake: loading data uses compute credits. Larger warehouses load data faster but cost more. Monitor load durations and size to choose warehouse size. Use `COPY` for bulk (lower cost/throughput) rather than row-by-row inserts.
- **Libraries & Versions:**
 - Spring Boot **3.x** / Spring Batch **6.x** (latest stable).
 - Spring Cloud AWS **3.x** (aws-spring integration, BOM) for S3 and Secrets.
 - AWS SDK for Java **2.x** (current v2.48+).
 - Spring Integration **7.x** for SFTP.
 - Quartz **2.6.x**.
 - Snowflake JDBC **3.x** or **4.x** (latest) driver.
 - Micrometer **latest** (for metrics).

Design Alternatives

Approach	Pros	Cons	Use Cases
Polling (Quartz)	Simple to implement; time-driven control (cron)	Latency (min delay = poll interval); wasted checks if no new files	Regular batch loads; predictable schedules

Approach	Pros	Cons	Use Cases
Event-driven	Real-time reaction (e.g. AWS S3 Event + Snowpipe); lower latency	More complex; need event infra (SNS/SQS/Lambda or AWS Transfer)	Near-real-time ETL; large event volumes
SFTP-Pull	Our own app retrieves files; complete control	Must handle connectivity/retries; time scheduling needed	When SFTP server is external and push not supported
SFTP-Push (AWS Transfer)	Offload transfer to AWS; event-driven copy to S3	Requires AWS Transfer config and fees; less control	When partner can push and AWS Transfer is allowed
Snowflake COPY	Bulk, parallel load; optimized for big data 【41†L275-L278】 【22†L267-L271】	Requires staging files; uses Snowflake credits (compute)	Bulk batch loads (tens of MB to GB)
Snowflake Snowpipe	Serverless, automatic on S3 events; continuous load	Higher cost per credit for streaming; requires S3 notifications/SQS	Streaming or near-real-time loading
Direct JDBC Insert	Low setup (just code); fits small loads	Very slow for large data; high transaction cost	Small volumes (<10k rows) or metadata inserts

Configuration Examples

Quartz/YAML:

```
spring.quartz.job-store-type=jdbc
spring.quartz.properties.org.quartz.threadPool.threadCount=3

schedule:
  sftpJob: "0 0/15 * * * ?"      # every 15 min
  snowflakeJob: "0 5/15 * * * ?" # offset slightly
```

SFTP Session Bean (Java):

```
@Bean
public SessionFactory<LsEntry> sftpSessionFactory(
    @Value("${sftp.host}") String host,
    @Value("${sftp.port}") int port,
    @Value("${sftp.user}") String user,
    @Value("${sftp.password}") String pass)
{
    DefaultSftpSessionFactory factory = new DefaultSftpSessionFactory(true);
    factory.setHost(host);
    factory.setPort(port);
    factory.setUser(user);
```

```

        factory.setPassword(pass);
        factory.setAllowUnknownKeys(true);
        return new CachingSessionFactory<>(factory);
    }

```

Spring Batch Job/Step (Java):

```

@Bean
public Job sftpToS3Job(JobBuilderFactory jb, StepBuilderFactory sbf) {
    return jb.get("SftpToS3Job")
        .start(sftpToS3Step(sbf))
        .build();
}

@Bean
public Step sftpToS3Step(StepBuilderFactory sbf) {
    return sbf.get("SftpToS3Step")
        .<String, String>chunk(1)
        .reader(sftpFileItemReader())
        .processor(sftpFileItemProcessor())
        .writer(s3ItemWriter())
        .faultTolerant()
        .retryLimit(3).retry(Exception.class)
        .build();
}

```

Quartz JobLauncher (Java):

```

public class QuartzJobLauncher implements Job {
    @Autowired private JobLauncher jobLauncher;
    @Autowired private Job sftpToS3Job;

    @Override
    public void execute(JobExecutionContext ctx) {
        try {
            JobParameters params = new JobParametersBuilder()
                .addLong("time", System.currentTimeMillis()).toJobParameters();
            jobLauncher.run(sftpToS3Job, params);
        } catch (Exception e) {
            throw new JobExecutionException(e);
        }
    }
}

```

This custom Quartz job uses Spring's `JobLauncher` to start the batch job with a unique timestamp parameter `【17†L237-L244】` .

Code Snippets

FlatFileItemReader from S3 (via Spring Cloud AWS `Resource`):

```
@Bean
public FlatFileItemReader<String> reader() {
    return new FlatFileItemReaderBuilder<String>()
        .name("s3Reader")
        .resource(new FileSystemResource("s3://" + bucket + "/input/" +
            fileName))
        .lineMapper(new DefaultLineMapper<>())
        .build();
}
```

Spring Cloud AWS allows `s3://` URLs as `Resource` [【10†L139-L148】](#) . Alternatively, use `S3Client.getObject` with `RequestBody.fromInputStream` .

S3 Multipart Upload (AWS SDK v2):

```
long fileSize = channel.lstat(remotePath).getSize();
try (InputStream is = channel.get(remotePath)) {
    PutObjectRequest req = PutObjectRequest.builder()
        .bucket(bucket).key(key).build();
    s3Client.putObject(req, RequestBody.fromInputStream(is, fileSize));
}
```

For larger files (≥ 100 MB) consider `S3TransferManager` for multipart parallel upload [【24†L25-L28】](#) [【10†L127-L130】](#) .

Snowflake COPY into (JDBC):

```
String copySql = String.format(
    "COPY INTO %s FROM '%s/%s' FILE_FORMAT = (TYPE=CSV COMPRESSION=GZIP)",
    tableName, stageName, fileName);
try (Statement stmt = snowflakeConn.createStatement()) {
    stmt.execute(copySql);
}
```

Before this, ensure the external stage (`@stageName`) is set up:

```
CREATE STAGE IF NOT EXISTS my_stage
URL='s3://my-data-bucket/input'
CREDENTIALS=(AWS_ROLE='arn:aws:iam::...:role/SnowflakeLoadRole');
```

This uses Snowflake's recommendation to load via an external stage [【41†L275-L278】](#) .

Tables: Alternative Designs

Scheduling

Polling (Quartz)	Event-driven (e.g. AWS Lambda/Snowpipe)
+ Simple cron control, existing infra	+ Real-time, no polling lag
– Wasted cycles if idle; latency up to interval	– More components (SNS, SQS, Lambdas); complexity
Use when loads are periodic and can tolerate some delay 【30†L31-L34】 .	Use when files arrive unpredictably or need immediate processing.

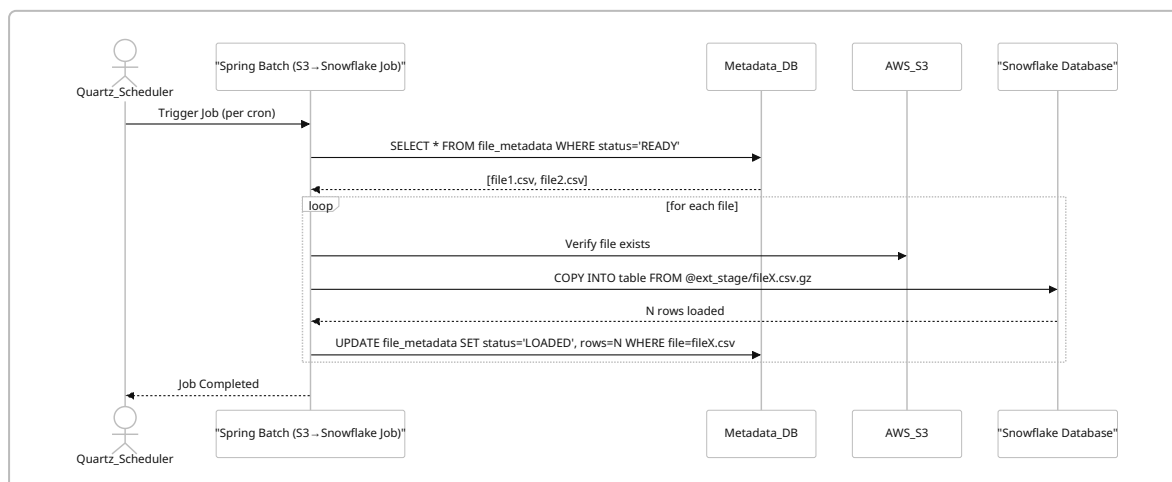
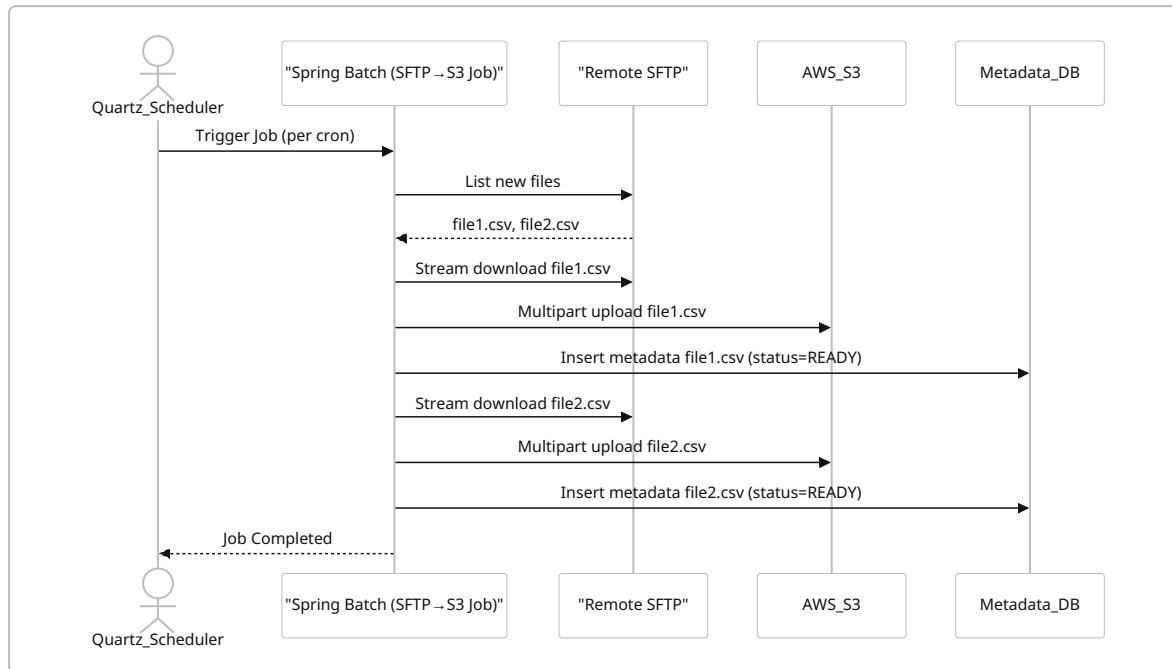
SFTP File Transfer

Pull (our app with JSch/Spring Integration)	Push (AWS Transfer Family SFTP connector)
+ Full control in app (retries, logging)	+ Managed by AWS, event-driven to S3
– Need to handle connectivity & schedule	– Requires AWS Transfer setup and cost
Standard approach if SFTP is external to AWS.	Good if partners can use AWS and event-based flows 【43†L11-L18】 .

Snowflake Ingestion

Staged COPY (recommended)	Snowpipe (event-driven continuous load)
+ High throughput (bulk) 【41†L275-L278】 , retries controlled	+ Automated on S3 events; low-latency
– Must run warehouse and manage loads	– Higher cost per credit; requires S3 event setup
Best for scheduled/batch loads.	Best for streaming or frequent small file loads.

Sequence Diagrams (Mermaid)



Recommended Libraries & Versions

- **Spring Boot:** 3.2+ (with Spring Batch 6.x)
- **Spring Batch:** 6.x (core ETL framework) [\[19†L138-L146\]](#)
- **Spring Cloud AWS:** 3.4+ (for S3 client, S3Resource, Secrets Manager) [\[10†L73-L81\]](#) [\[37†L39-L48\]](#)
- **AWS SDK for Java 2.x:** 2.48.x (including `software.amazon.awssdk:s3` and `s3-transfer-manager`) [\[10†L73-L81\]](#) [\[10†L127-L130\]](#)
- **Spring Integration SFTP:** 7.1+ (uses Apache MINA SSHD) [\[39†L369-L378\]](#)
- **Quartz Scheduler:** 2.6.x (via `spring-boot-starter-quartz`) [\[30†L31-L39\]](#)
- **Snowflake JDBC Driver:** 3.x or 4.x (latest)

Most recent versions should be managed via BOMs (Spring Boot dependency management, AWS SDK BOM [\[8†L339-L348\]](#)).

Snowflake Bulk Load Patterns

Use **staging and COPY**: create a named external stage for the S3 bucket. Then:

```
COPY INTO MY_TABLE
FROM @my_stage/path/ FILE_FORMAT = (TYPE=CSV COMPRESSION=GZIP);
```

Snowflake advises parallel bulk loads with ~100–250 MB compressed files [22†L267-L271] . For example, if CSV is large, gzip it and ensure files are in that optimal size range. Multi-file loads run in parallel on the warehouse. Use `COPY` options like `ON_ERROR= 'ABORT_STATEMENT'` to fail fast on data errors, or `CONTINUE` to skip bad rows. Monitor load via Snowflake's `INFORMATION_SCHEMA.LOAD_HISTORY` . Optionally use **Streams/Tasks** for incremental loads.

S3 Transfer Best Practices

- **Multipart Upload**: Use multipart for any object >100 MB [24†L25-L28] . The AWS SDK v2 `S3TransferManager` handles this automatically with async parallel parts.
- **Encryption**: Enable bucket encryption (SSE-S3 or SSE-KMS). AWS recommends SSE-KMS for flexibility [25†L169-L174] . Also encrypt data in transit (HTTPS).
- **Bandwidth**: For large files, use concurrent threads (SDK's `TransferManager`) to maximize throughput [24†L25-L28] .
- **Retries**: AWS clients automatically retry idempotently. Configure max retry and backoff (via `ClientConfiguration` or `AwsClientBuilder` options).
- **Cleanup**: Optionally set lifecycle rules to archive or delete old files in S3 to reduce cost.

Cost & Throughput Considerations

- **AWS S3**: Cost = storage + PUT/GET requests. Minimize small files to reduce request overhead. Use Infrequent Access or Glacier for archival data if needed.
- **Snowflake**: Loading consumes compute (credits). A larger warehouse loads faster but costs more. Use auto-suspend to avoid idle cost.
- **Throughput**: Network and disk IO are factors. In Spring Batch, tuning chunk size and thread count can help. For Snowflake, parallel `COPY` uses all nodes in the warehouse. Monitor load duration and scale the warehouse accordingly.
- **Costs**: Keep intermediate data minimal. Clean S3 staging area after load if not needed. Consider incremental vs full loads to reduce processed data.

Citations

- Spring Cloud AWS auto-configures an `S3Client` bean and allows accessing S3 objects via `s3://bucket/object` URLs as Spring `Resource`'s [10†L73-L81] [10†L139-L148] .
- AWS recommends multipart upload for objects ≥ 100 MB to improve throughput and resiliency [24†L25-L28] .
- Snowflake suggests staging files in S3 and using `COPY INTO` (preferably via an external stage) for bulk loads [41†L275-L278] , with ideal file sizes ~100–250 MB [22†L267-L271] .
- Spring Boot's Quartz integration auto-configures a `Scheduler` if `spring-boot-starter-quartz` is present [30†L31-L39] .

- Spring Batch provides built-in metrics (`spring.batch.job` , `spring.batch.step`) via Micrometer 【33†L172-L180】 .
 - AWS Transfer Family SFTP Connectors can retrieve files from external SFTP to S3 in an event-driven way, enabling automated workflows 【43†L11-L18】 .
 - Spring Integration 7.x's SFTP support (Apache MINA SSHD) can be used for reliable SFTP transfers 【39†L369-L378】 .
-